

# Mini Job Board: Full System Documentation

## 1. Project Overview

The objective of the Mini Job Board is to design and develop a full-stack, role-based application bridging student candidates and employers. It serves as a modern, simplified version of LinkedIn with a focus on responsive dashboards, job search functionality, and candidate profile customization.

### Target User Personas

The system utilizes two distinct roles with restricted API permissions and custom layouts:

- **Candidate Persona:** Students seeking entry-level positions or internships who can search, filter, bookmark, and track job applications in real-time.
- **Employer Persona:** Recruiters who post job opportunities, manage active listings, and review incoming candidate credentials and resumes.

## 2. System Architecture

The application uses a Client-Server Architecture with a stateless, RESTful backend and a component-driven Single Page Application (SPA) frontend.

### Architectural Layers

- **Presentation Layer (Controllers):** Exposes HTTP endpoints, validates incoming DTOs, and handles role-based authorization using `@PreAuthorize`.
- **Business Logic Layer (Services):** Implements core workflows and prevents IDOR attacks by verifying object ownership before deletions.
- **Data Access Layer (Repositories):** Utilizes Spring Data JPA for relational access and native custom queries for performance.
- **Security Layer:** Implements stateless JWT authorization to validate request signatures before controller access.

### 3. Technical Stack

| Component  | Technology       | Version | Description            |
|------------|------------------|---------|------------------------|
| Backend    | Java SDK         | 17      | Core language platform |
| Framework  | Spring Boot      | 3.5.0   | Web, Security, JPA     |
| Database   | MySQL            | 8.0+    | Relational storage     |
| Frontend   | React            | 19.2.4  | UI framework           |
| Routing    | React Router DOM | 7.14.0  | Page navigation        |
| Build Tool | Vite             | 8.0.1   | Bundler                |

### 4. Data Model Design

The relational schema is built on MySQL and managed via JPA annotations.

#### Core Database Tables

- **users:** Stores credentials and profile metadata (headline, location, avatar\_url) for both candidates and employers.
- **jobs:** Contains listings posted by employers, including workplace\_type (Remote, Hybrid, Onsite) and employment\_type.
- **applications:** Tracks job submissions with status fields (Pending, Accepted, Rejected).
- **experiences / educations:** Details professional and academic history for candidates.
- **user\_saved\_jobs:** A join table managing the many-to-many relationship for bookmarked postings.

### 5. Security and State Management

Authentication is completely stateless, utilizing BCrypt for password hashing and JWT (0.12.5) for token management.

## Authentication Flow

1. **Login:** User credentials are sent to `/api/auth/login`.
2. **JWT Generation:** A cryptographically signed token is generated and returned to the client.
3. **Storage:** The frontend saves the token in `localStorage`.
4. **Authorization:** Subsequent requests include the token in the `Authorization: Bearer <TOKEN>` header.

## 6. Development Roadmap (12 Weeks)

The project was executed across 12 weekly cycles:

- **Week 1-2:** Kick-off and Requirements Analysis (User stories and MVP scope).
- **Week 3-5:** System Design and Technical Setup (Architecture, Spring Boot skeleton, and ERD design).
- **Week 6-8:** Core Logic and UI Development (CRUD API, React layouts, and system integration).
- **Week 9-10:** Security and Testing (Spring Security, JWT integration, and Mockito unit tests).
- **Week 11-12:** Documentation and Final Presentation.

## 7. Installation and Setup

### Prerequisites

- Java JDK 17+.
- Node.js (v16.x+) and npm.
- MySQL Server on port 3306.

### Execution Commands

- **Backend:** Navigate to `backend` and run `./mvnw spring-boot:run` (macOS/Linux) or `mvnw.cmd spring-boot:run` (Windows).
- **Frontend:** Navigate to `frontEnd`, run `npm install`, then `npm run dev` to start the server at `http://localhost:5173`.
- **Tests:** Execute backend unit tests using `./mvnw test`.

---